

Managing a PyTorch Training Process with Checkpoints and Early Stopping

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch**  4

[Share](#) [Share](#) [Post](#)

A large deep learning model can take a long time to train. You lose a lot of work if the training process interrupted in the middle. But sometimes, you actually want to interrupt the training process in the middle because you know going any further would not give you a better model. In this post, you will discover how to control the training loop in PyTorch such that you can resume an interrupted process, or early stop the training loop.

After completing this post, you will know:

- The importance of checkpointing neural network models when training
- How to checkpoint a model during training and restore it later
- How to terminate training loop early with checkpointing

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Managing a PyTorch Training Process with Checkpoints and Early Stopping
Photo by Arron Choi. Some rights reserved.

Overview

This chapter is in two parts; they are:

- Checkpointing Neural Network Models
- Checkpointing with Early Stopping

Checkpointing Neural Network Models

A lot of systems have states. If you can save all its state from a system and restore it later, you can always move back in a particular point in time about how a system behaves. If you worked on Microsoft Word and saved multiple versions of a document because you don't know if you want to revert back your edit, it is the same idea here.

Same applies to long-running processes. Application checkpointing is a fault tolerance technique. In this approach, a snapshot of the state of the system is taken in case of system failure. If there is a problem, you can resume from the snapshot. The checkpoint may be used directly or as the starting point for a new run, picking up where it left off. When training deep learning models, the checkpoint captures the weights of the model. These weights can be used to make predictions as-is or as the basis for ongoing training.

PyTorch does not provide any function for checkpointing but it has functions for retrieving and restoring weights of a model. So you can implement checkpointing logic with them. Let's make a checkpoint and a resume function, which simply save weights from a model and load them back:

```
1 import torch
2
3 def checkpoint(model, filename):
4     torch.save(model.state_dict(), filename)
5
6 def resume(model, filename):
7     model.load_state_dict(torch.load(filename))
```

Below is how you would usually do to train a PyTorch model. The dataset used is fetched from OpenML platform. It is a binary classification dataset. PyTorch DataLoader is used in this example to make the training loop more concise.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import TensorDataset, DataLoader, random_split, default_collate
5 from sklearn.datasets import fetch_openml
6 from sklearn.preprocessing import LabelEncoder
7
8 data = fetch_openml("electricity", version=1, parser="auto")
9
10 # Label encode the target, convert to float tensors
11 X = data['data'].astype('float').values
12 y = data['target']
13 encoder = LabelEncoder()
14 encoder.fit(y)
15 y = encoder.transform(y)
16 X = torch.tensor(X, dtype=torch.float32)
17 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
18
19 # train-test split for model evaluation
20 trainset, testset = random_split(TensorDataset(X, y), [0.7, 0.3])
21
22 # Define the model
23 model = nn.Sequential(
24     nn.Linear(8, 12),
25     nn.ReLU(),
26     nn.Linear(12, 12),
27     nn.ReLU(),
28     nn.Linear(12, 1),
29     nn.Sigmoid(),
30 )
31
32 # Train the model
33 n_epochs = 100
34 loader = DataLoader(trainset, shuffle=True, batch_size=32)
35 X_test, y_test = default_collate(testset)
36 loss_fn = nn.BCELoss()
37 optimizer = optim.SGD(model.parameters(), lr=0.1)
38 for epoch in range(n_epochs):
39     model.train()
40     for X_batch, y_batch in loader:
41         y_pred = model(X_batch)
42         loss = loss_fn(y_pred, y_batch)
43         optimizer.zero_grad()
44         loss.backward()
45         optimizer.step()
46     model.eval()
47     y_pred = model(X_test)
48     acc = (y_pred.round() == y_test).float().mean()
49     print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
```

If you want to add checkpoints to the training loop above, you can do it at the end of the outer for-loop, where the model validation with the test set is done. Let's say, the following:

```
1 ...
2 for epoch in range(n_epochs):
3     model.train()
4     for X_batch, y_batch in loader:
5         y_pred = model(X_batch)
6         loss = loss_fn(y_pred, y_batch)
7         optimizer.zero_grad()
8         loss.backward()
9         optimizer.step()
```

```

10 model.eval()
11 y_pred = model(X_test)
12 acc = (y_pred.round() == y_test).float().mean()
13 print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
14 checkpoint(model, f"epoch-{epoch}.pth")

```

You will see a number of files created in your working directory. This code is going to checkpoint the model from epoch 7, for example, into file epoch-7.pth. Each of these file is a ZIP file with the pickled model weight. Nothing forbid you to checkpoint inside the inner for-loop but due to the overhead it incurs, it is not a good idea to checkpoint too frequent.

As a fault tolerance technique, by adding a few lines of code before the training loop, you can resume from a particular epoch:

```

1 start_epoch = 0
2 if start_epoch > 0:
3     resume_epoch = start_epoch - 1
4     resume(model, f"epoch-{resume_epoch}.pth")
5
6 for epoch in range(start_epoch, n_epochs):
7     model.train()
8     for X_batch, y_batch in loader:
9         y_pred = model(X_batch)
10        loss = loss_fn(y_pred, y_batch)
11        optimizer.zero_grad()
12        loss.backward()
13        optimizer.step()
14    model.eval()
15    y_pred = model(X_test)
16    acc = (y_pred.round() == y_test).float().mean()
17    print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
18    checkpoint(model, f"epoch-{epoch}.pth")

```

That is, if the training loop was interrupted in the middle of epoch 8 so the last checkpoint is from epoch 7, setting start_epoch = 8 above will do.

Note that if you do so, the random_split() function that generate the training set and test set may give you different split due to the random nature. If that's a concern for you, you should have a consistent way of creating the datasets (e.g., save the splitted data so you can reuse them).

Sometimes, there are states outside of the model and you may want to checkpoint it as well. One particular example is the optimizer, which in cases like Adam, there are dynamically adjusted momentum. If you restarted your training loop, you may want to restore the momentum at the optimizer as well. It is not difficult to do. The idea is to make your checkpoint() function more complicated, e.g.

```

1 torch.save({
2     'optimizer': optimizer.state_dict(),
3     'model': model.state_dict(),
4 }, filename)

```

and correspondingly, change your resume() function:

```

1 checkpoint = torch.load(filename)
2 model.load_state_dict(checkpoint['model'])
3 optimizer.load_state_dict(checkpoint['optimizer'])

```

This works because in PyTorch, the torch.save() and torch.load() function are backed by pickle, so you can use it with a list or dict container.

To put everything together, below is the complete code:

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import TensorDataset, DataLoader, random_split, default_collate
5 from sklearn.datasets import fetch_openml
6 from sklearn.preprocessing import LabelEncoder
7
8 data = fetch_openml("electricity", version=1, parser="auto")
9
10 # Label encode the target, convert to float tensors
11 X = data['data'].astype('float').values
12 y = data['target']
13 encoder = LabelEncoder()
14 encoder.fit(y)
15 y = encoder.transform(y)
16 X = torch.tensor(X, dtype=torch.float32)
17 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
18
19 # train-test split for model evaluation
20 trainset, testset = random_split(TensorDataset(X, y), [0.7, 0.3])
21
22 def checkpoint(model, filename):
23     torch.save(model.state_dict(), filename)
24
25 def resume(model, filename):
26     model.load_state_dict(torch.load(filename))
27
28 # Define the model
29 model = nn.Sequential(
30     nn.Linear(8, 12),
31     nn.ReLU(),
32     nn.Linear(12, 12),

```

```

33     nn.ReLU(),
34     nn.Linear(12, 1),
35     nn.Sigmoid(),
36 )
37
38 # Train the model
39 n_epochs = 100
40 start_epoch = 0
41 loader = DataLoader(trainset, shuffle=True, batch_size=32)
42 X_test, y_test = default_collate(testset)
43 loss_fn = nn.BCELoss()
44 optimizer = optim.SGD(model.parameters(), lr=0.1)
45
46 if start_epoch > 0:
47     resume_epoch = start_epoch - 1
48     resume(model, f"epoch-{resume_epoch}.pth")
49
50 for epoch in range(start_epoch, n_epochs):
51     model.train()
52     for X_batch, y_batch in loader:
53         y_pred = model(X_batch)
54         loss = loss_fn(y_pred, y_batch)
55         optimizer.zero_grad()
56         loss.backward()
57         optimizer.step()
58     model.eval()
59     y_pred = model(X_test)
60     acc = (y_pred.round() == y_test).float().mean()
61     print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
62     checkpoint(model, f"epoch-{epoch}.pth")

```

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Checkpointing with Early Stopping

Checkpointing is not only for fault tolerance. You can also use it to keep your best model. How to define what is the best is subjective but considering the score from the test set is a sensible method. Let's say to keep only the best model ever found, you can modify the training loop as follows:

```

1  ...
2  best_accuracy = -1
3  for epoch in range(start_epoch, n_epochs):
4      model.train()
5      for X_batch, y_batch in loader:
6          y_pred = model(X_batch)
7          loss = loss_fn(y_pred, y_batch)
8          optimizer.zero_grad()
9          loss.backward()
10         optimizer.step()
11     model.eval()
12     y_pred = model(X_test)
13     acc = (y_pred.round() == y_test).float().mean()
14     acc = float(acc) * 100
15     print(f"End of epoch {epoch}: accuracy = {acc:.2f}%")
16     if acc > best_accuracy:
17         best_accuracy = acc
18         checkpoint(model, "best_model.pth")
19
20 resume(model, "best_model.pth")

```

The variable `best_accuracy` is to keep track on the highest accuracy (`acc`) obtained so far, which is in a percentage range of 0 to 100. Whenever a higher accuracy is observed, the model is checkpointed to the file `best_model1.pth`. The best model is restored after the entire training loop, via the `resume()` function you created before. Afterward, you can make predictions with the model on unseen data. Beware that, if you're using a different metric for checkpointing, e.g., the cross entropy loss, the better model should come with a lower cross entropy. Thus you should keep track on the lowest cross entropy obtained.

You can also checkpoint the model per epoch unconditionally together with the best model checkpointing, as you are free to create multiple checkpoint files. Since the code above is the find the best model and make a copy of it, you may usually see a further optimization to the training loop by stopping it early if the hope to see model improvement is slim. This is the early stopping technique that can save time in training.

The code above validates the model with test set at the end of each epoch and keeps the best model found into a checkpoint file. The simplest strategy for early stopping is to set up a threshold of k epochs. If you didn't see the model improved over the last k epochs, you terminate the training loop in the

middle. This can be implemented as follows:

```

1  early_stop_thresh = 5
2  best_accuracy = -1
3  best_epoch = -1
4  for epoch in range(start_epoch, n_epochs):
5      model.train()
6      for X_batch, y_batch in loader:
7          y_pred = model(X_batch)
8          loss = loss_fn(y_pred, y_batch)
9          optimizer.zero_grad()
10         loss.backward()
11         optimizer.step()
12     model.eval()
13     y_pred = model(X_test)
14     acc = (y_pred.round() == y_test).float().mean()
15     acc = float(acc) * 100
16     print(f"End of epoch {epoch}: accuracy = {acc:.2f}%")
17     if acc > best_accuracy:
18         best_accuracy = acc
19         best_epoch = epoch
20         checkpoint(model, "best_model.pth")
21     elif epoch - best_epoch > early_stop_thresh:
22         print("Early stopped training at epoch %d" % epoch)
23         break # terminate the training loop
24
25 resume(model, "best_model.pth")

```

The threshold `early_stop_thresh` was set to 5 above. There is a variable `best_epoch` that remembers the epoch of the best model. If the model has not been improved for long enough, the outer for-loop will be terminated.

This design is a relief on one of the design parameter, `n_epochs`. You can now make `n_epochs` the **maximum** number of epochs to train the model, hence a larger number than needed and assured that usually your training loop will stop earlier. This is also a strategy to avoid overfitting: If the model is indeed perform worse as you trained it further on the test set, this early stopping logic will interrupt the training and restore the best checkpoint.

Tying everything together, the following is the complete code for checkpointing with early stopping:

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import TensorDataset, DataLoader, random_split, default_collate
5  from sklearn.datasets import fetch_openml
6  from sklearn.preprocessing import LabelEncoder
7
8  data = fetch_openml("electricity", version=1, parser="auto")
9
10 # Label encode the target, convert to float tensors
11 X = data['data'].astype('float').values
12 y = data['target']
13 encoder = LabelEncoder()
14 encoder.fit(y)
15 y = encoder.transform(y)
16 X = torch.tensor(X, dtype=torch.float32)
17 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
18
19 # train-test split for model evaluation
20 trainset, testset = random_split(TensorDataset(X, y), [0.7, 0.3])
21
22 def checkpoint(model, filename):
23     torch.save(model.state_dict(), filename)
24
25 def resume(model, filename):
26     model.load_state_dict(torch.load(filename))
27
28 # Define the model
29 model = nn.Sequential(
30     nn.Linear(8, 12),
31     nn.ReLU(),
32     nn.Linear(12, 12),
33     nn.ReLU(),
34     nn.Linear(12, 1),
35     nn.Sigmoid(),
36 )
37
38 # Train the model
39 n_epochs = 10000 # more than we needed
40 loader = DataLoader(trainset, shuffle=True, batch_size=32)
41 X_test, y_test = default_collate(testset)
42 loss_fn = nn.BCELoss()
43 optimizer = optim.SGD(model.parameters(), lr=0.1)
44
45 early_stop_thresh = 5
46 best_accuracy = -1
47 best_epoch = -1
48
49 for epoch in range(n_epochs):
50     model.train()
51     for X_batch, y_batch in loader:
52         y_pred = model(X_batch)
53         loss = loss_fn(y_pred, y_batch)
54         optimizer.zero_grad()
55         loss.backward()

```

```

56     optimizer.step()
57     model.eval()
58     y_pred = model(X_test)
59     acc = (y_pred.round() == y_test).float().mean()
60     acc = float(acc) * 100
61     print(f"End of epoch {epoch}: accuracy = {acc:.2f}%")
62     if acc > best_accuracy:
63         best_accuracy = acc
64         best_epoch = epoch
65         checkpoint(model, "best_model.pth")
66     elif epoch - best_epoch > early_stop_thresh:
67         print("Early stopped training at epoch %d" % epoch)
68         break # terminate the training loop
69
70 resume(model, "best_model.pth")

```

You may see the above code to produce:

```

1 End of epoch 0: accuracy = 61.84%
2 End of epoch 1: accuracy = 55.90%
3 End of epoch 2: accuracy = 63.95%
4 End of epoch 3: accuracy = 66.87%
5 End of epoch 4: accuracy = 64.77%
6 End of epoch 5: accuracy = 60.03%
7 End of epoch 6: accuracy = 67.16%
8 End of epoch 7: accuracy = 66.01%
9 End of epoch 8: accuracy = 62.88%
10 End of epoch 9: accuracy = 64.28%
11 End of epoch 10: accuracy = 68.63%
12 End of epoch 11: accuracy = 70.56%
13 End of epoch 12: accuracy = 64.62%
14 End of epoch 13: accuracy = 65.63%
15 End of epoch 14: accuracy = 66.81%
16 End of epoch 15: accuracy = 65.11%
17 End of epoch 16: accuracy = 55.81%
18 End of epoch 17: accuracy = 54.59%
19 Early stopped training at epoch 17

```

It stopped at end of epoch 17 for the best model obtained from epoch 11. Due to the stochastic nature of algorithm, you may see the result slightly different. But for sure, even when the maximum number of epochs set to 10000 above, the training loop indeed stopped much earlier.

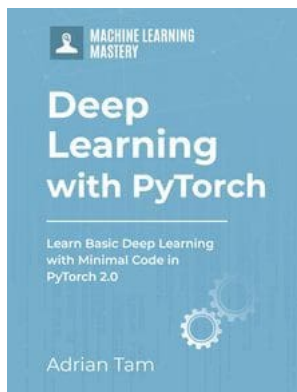
Of course, you can design a more sophisticated early stopping strategy, e.g., run for at least N epochs and then allow to early stop after k epochs. You have all the freedom to tweak the code above to make the best training loop to fit your need.

Summary

In this chapter, you discovered the importance of checkpointing deep learning models for long training runs. You learned:

- What is checkpointing and why it is useful
- How to checkpoint your model and how to restore the checkpoint
- Different strategies to use checkpoints
- How to implement early stopping with checkpointing

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

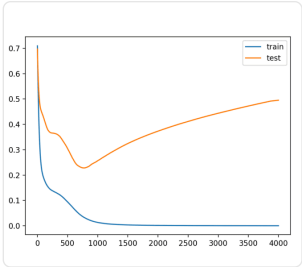
SEE WHAT'S INSIDE

Share

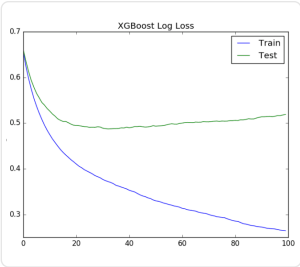
Share

Post

More On This Topic



Use Early Stopping to Halt the Training of Neural...



Avoid Overfitting By Early Stopping With XGBoost In Python



A Gentle Introduction to Early Stopping to Avoid...



Applied Machine Learning Process



Managing Data for Machine Learning Projects



Understanding RAG Part V: Managing Context Length



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

< Understand Model Behavior During Training by Visualizing Metrics Visualizing a PyTorch Model >

4 Responses to Managing a PyTorch Training Process with Checkpoints and Early Stopping



AM September 4, 2023 at 8:12 am #

REPLY ↩

Thank you. Any articles that you know of where checkpoint is used to save some memory and allow for additional gradient accumulation steps?



James Carmichael September 5, 2023 at 6:37 am #

REPLY ↩

Hi AM...The following discussion may be of interest:

<https://ai.stackexchange.com/questions/31675/what-is-better-to-use-early-stopping-model-checkpoint-or-both>



Rodrigo October 3, 2024 at 9:11 am #

REPLY ↩

Thank you for the tutorial.

I am curious where torch.utils.checkpoint comes into play with respect to all this.

Small feedback: writing

```
start_epoch = 0
if start_epoch > 0:
```

is a bit confusing since at first glance the if condition will never be satisfied. I guess the intention is that the user should change the initial value 0 to another value they desire to start from, but this intention is not very evident. I would advise either making that a command-line argument, or at least adding a comment: # change this value to desired starting epoch.



Rodrigo October 3, 2024 at 9:17 am #

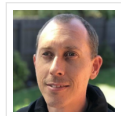
REPLY ↩

Ok, never mind, after some searching I see torch.utils.checkpoint serves a completely different purpose, namely saving GPU memory by discarding some values and recomputing them when needed. An unfortunate name choice if you ask me.

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)

[Calculating Derivatives in PyTorch](#)[Develop Your First Neural Network with PyTorch, Step by Step](#)

Loving the Tutorials?

The Deep Learning with PyTorch EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved